MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 3

Software Design

Author: Iulia Maria Martin
Group: 30433

Faculty of Automation and Computer Science

March, 2026

Contents

0.1 Problem definition 2

1 Analysis phase - Use cases & requirements 2

1.1 Functional requirements 2

1.2 Use cases 3

2 Design Phase 4

2.1 Architecture Overview 4

2.2 Entity-Relationship Diagram 5

2.3 Component X - API Gateway (Frontend) 5

2.4 Component Y - Microservices Backend 6

2.5 Design Patterns 6

2.6 Class Diagrams 6

2.7 Sequence Diagrams 7

3 Implementation Phase - Application 7

3.1 Tools and Technologies 7

3.2 Application Structure 8

3.3 Testing and Validation 8

0.1 Problem definition

This project extends the Flower Shop Management System developed in Assignments 1 and 2 by evolving it from a monolithic event-driven application into a highly scalable, distributed **Microservices-based architecture**. The application continues to manage the complete workflow of a flower retail business, but its core domains (Catalog, Orders, Users) have been entirely decoupled into independent Web APIs. Furthermore, the system introduces the **Command-Query Responsibility Segregation (CQRS)** architectural pattern, refactors data export using the **Template Method** design pattern, and transitions the legacy MVC application into a **Hybrid API Gateway**.

Development Phases

The project was extended in four major phases: Phase 1 focused on **Microservices Separation**, breaking the monolithic backend into three standalone ASP.NET Core Web API projects (CatalogService, OrderService, UserService) and a shared Core library. Phase 2 introduced the **CQRS and Command Pattern**, strictly separating database read operations (Queries) from write operations (Commands) across all microservices using explicit Dispatchers and Handlers. Phase 3 refactored the legacy Strategy export pipeline into the **Template Method pattern**, centralizing the algorithm skeleton in an abstract base class. Phase 4 achieved **Frontend Integration (Option B)**, transforming the original MVC application into an API Gateway that routes user actions via `HttpClient` to the respective microservices.

Additional Features and Deliverables

The system now includes three autonomous, self-hosted Web APIs running on independent ports. The MVC frontend acts as a Backend-For-Frontend (BFF) gateway. Design patterns are explicitly applied: **Command** (encapsulated in CQRS), **Template** (for data export), along with legacy patterns like Singleton and Observer for fallback operations. The entire CQRS pipeline and Template pattern are validated by a robust suite of **xUnit** automated tests using Entity Framework's InMemory database provider.

1 Analysis phase - Use cases & requirements

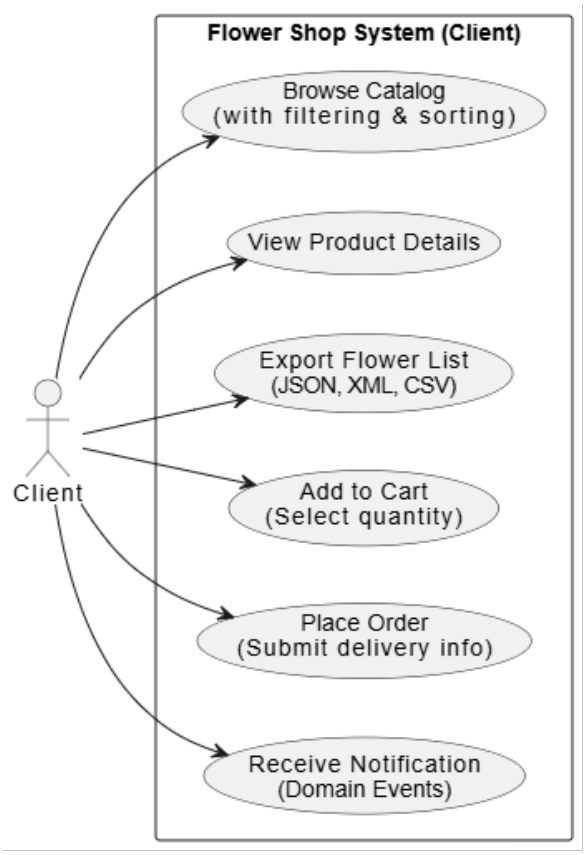
1.1 Functional requirements

The system fully maintains all Assignment 1 and 2 functional specifications while fundamentally shifting the execution context. When a user interacts with the UI (e.g., clicking "Add Flower" or "Place Order"), the MVC application no longer accesses the database directly for core domains. Instead, it serializes the action into a Command object and delegates it via HTTP JSON requests to the specific Microservice API. The Microservice decodes the payload, routes it through the CQRS `CommandDispatcher`, executes the business logic, and persists the data. Reading data follows a parallel path via the `QueryDispatcher`. Data export is handled locally by the Gateway using the newly implemented Template pattern, ensuring maximum reuse of serialization logic.

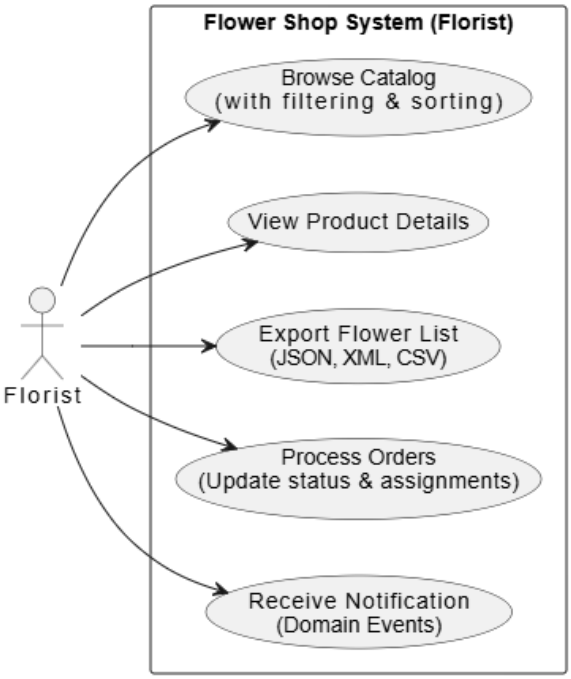
1.2 Use cases

Use cases describe in text the interactions between actors and the system in order to accomplish specific tasks. The user experience remains seamless, abstracting the distributed backend complexity.

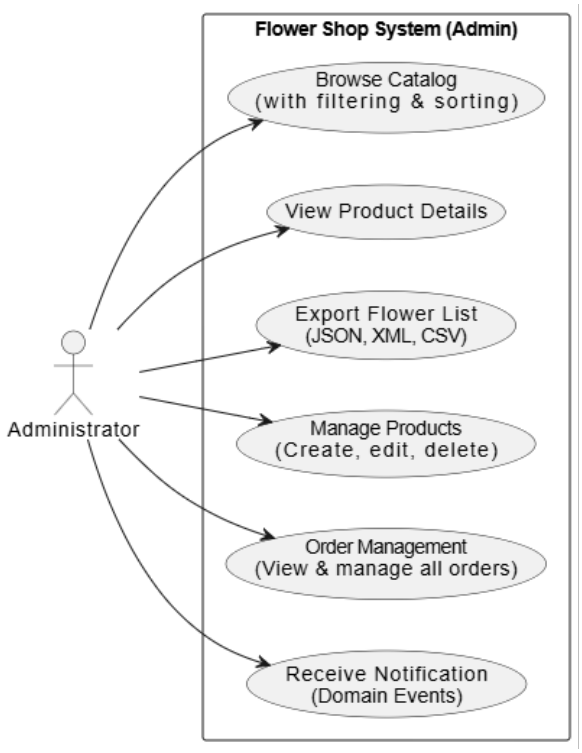
- **Browse Catalog** - Visitor views flowers; Gateway routes a `GetAllFlowersQuery` to CatalogService.
- **View Product Details** - Client views details; Gateway routes a `GetFlowerByIdQuery` to CatalogService.
- **Export Flower List** - Any user downloads the catalog in JSON, XML, or CSV via the Template Pattern.
- **Add to Cart** - Client adds items to the local shopping cart.
- **Place Order** - Client confirms cart; Gateway issues a `CreateOrderCommand` to OrderService.
- **Manage Products** - Admin creates/edits flowers; Gateway issues a `CreateFlowerCommand` to CatalogService.
- **User Authentication** - Visitor logs in; Gateway validates credentials using legacy fallback.
- **User Management** - Admin views users; Gateway issues `GetUserByIdQuery` to UserService.



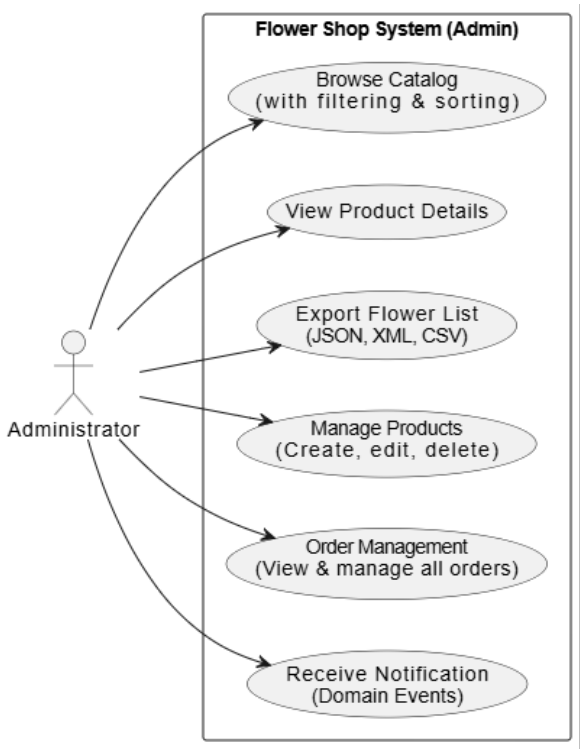
(a) Visitor Diagram



(b) Client Diagram



(a) Florist Diagram



(b) Admin Diagram

2 Design Phase

2.1 Architecture Overview

The application has successfully evolved from a monolithic structure into a **Distributed Microservices Architecture** utilizing the **API Gateway** pattern:

- **FlowerShop.CatalogService** — Independent Web API port (5001) dedicated exclusively to flower and category management.
- **FlowerShop.OrderService** — Independent Web API port (5002) dedicated to checkout processes, carts, and order history.
- **FlowerShop.UserService** — Independent Web API port (5003) managing user profiles and roles.
- **FlowerShop.Core** — Shared class library containing common CQRS interfaces, Domain Models, and the Template export logic, referenced by all microservices to ensure DRY principles.
- **Software_Design1 (API Gateway)** — The legacy ASP.NET Core MVC application stripped of its core database responsibilities. It now acts as a routing layer, using `IHttpClientFactory` to orchestrate HTTP calls to the backend microservices while preserving the UI rendering and user session state.

Internally, each Microservice implements the **CQRS Pattern**. API Controllers do not contain business logic; they only instantiate Command or Query records and pass them to an

`ICommandDispatcher` or `IQueryDispatcher`. This ensures strict decoupling and high cohesiveness within the domain layer.

2.2 Entity-Relationship Diagram

The domain model from the previous assignments is unchanged physically, but its logical access boundaries are now distributed.

Catalog Domain: Flowers, Categories, Bouquets, and Bouquet_Flowers tables are primarily owned and mutated by the CatalogService.

Order Domain: Orders and Order_Items tables are owned by the OrderService.

User Domain: Users and Florists tables are managed by the UserService.

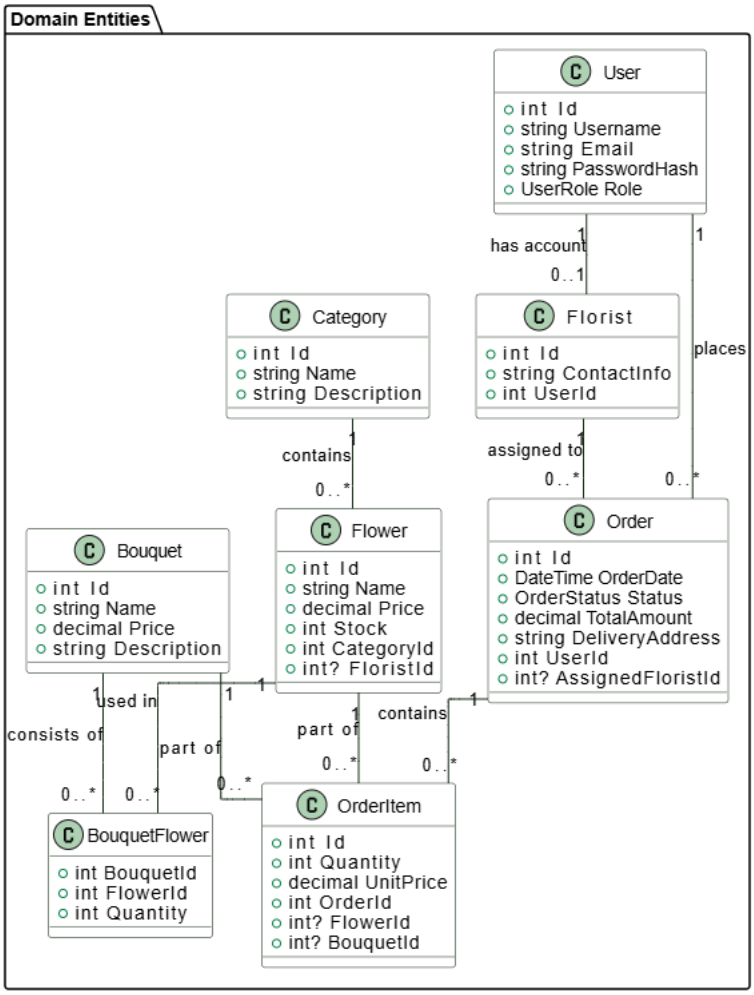


Figure 3: Entity-Relationship Diagram

2.3 Component X - API Gateway (Frontend)

The former monolithic web application server has been transformed into a **Hybrid API Gateway**. While it still handles session middleware and HTML rendering via Razor Views,

its internal `Service Layer` (e.g., `FlowerService`, `OrdersService`) has been entirely refactored. Instead of invoking Entity Framework `DbContext` operations, these services now inject `IHttpClientFactory` and issue asynchronous JSON HTTP requests to the respective microservices. For edge cases and legacy functions, it maintains a fallback database connection, ensuring uninterrupted UI functionality during the architectural transition.

2.4 Component Y - Microservices Backend

The backend consists of three independent ASP.NET Core Web API processes. Each microservice registers its own `AppDbContext` connection to MySQL and exposes stateless RESTful endpoints. The microservices configure `System.Text.Json` with `ReferenceHandler.IgnoreCycles` to gracefully handle Entity Framework object graph serialization without infinite loops.

2.5 Design Patterns

The system explicitly implements the following design patterns:

- **CQRS (Architectural)**: Segregates operations that read data (Queries) from operations that write data (Commands). This maximizes performance, scalability, and security by isolating the two concerns.
- **Command Pattern** (Behavioral): Central to the CQRS implementation. Every write action is encapsulated in a standalone class (e.g., `CreateFlowerCommand`). This object contains all necessary parameters to perform the action, decoupling the invoker (Controller/Dispatcher) from the receiver (CommandHandler).
- **Template Method** (Behavioral): Refactored from the previous Strategy pattern. The `DataExporter<T>` abstract base class defines the invariant algorithm skeleton for exporting data (`Export` method calling `FetchData`, `TransformData`, `WriteOutput`). Subclasses like `JsonExporter` and `XmlExporter` override only the primitive `TransformData` step, heavily reducing code duplication.
- **API Gateway** (Architectural): The MVC application acts as a single point of entry for the client, routing requests to multiple backend microservices, aggregating results, and handling cross-cutting concerns like authorization.

2.6 Class Diagrams

Class diagrams describe the static structure of the newly implemented subsystems.

The **CQRS subsystem** introduces `ICommand<TResult>`, `ICommandHandler<TCommand, TResult>`, and `ICommandDispatcher`. The `CreateFlowerCommand` implements `ICommand<int>`. The `CommandDispatcher` receives this command and uses `IServiceProvider` reflection to dynamically resolve and invoke `CreateFlowerCommandHandler`. A parallel structure exists for Queries (`IQuery`, `IQueryHandler`, `QueryDispatcher`).

The **Template Pattern subsystem** introduces the abstract `DataExporter<T>` class featuring the non-virtual `Export` template method and the abstract `TransformData` method. Concrete classes `JsonExporter`, `XmlExporter`, and `CsvExporter` inherit from the base class and implement the format-specific transformation logic.

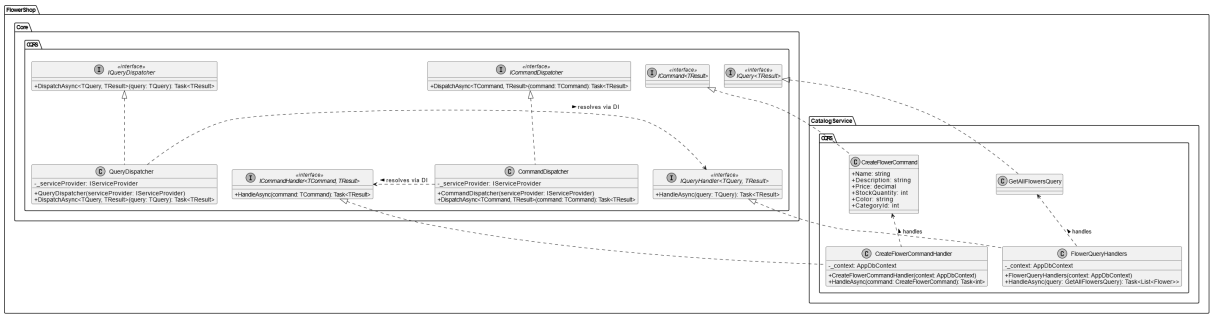


Figure 4: Class Diagram — CQRS and Command Pattern

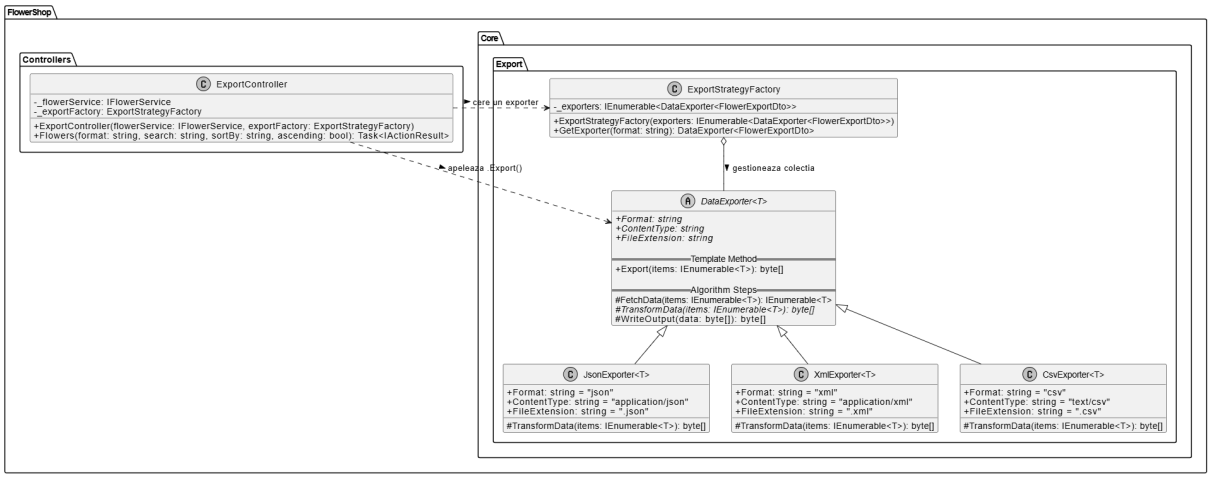


Figure 5: Class Diagram — Template Pattern

2.7 Sequence Diagrams

Sequence diagrams illustrate how objects interact during the execution of a particular use case.

The **Distributed CQRS sequence** shows the new workflow: The Admin submits a Create Flower form. The Gateway's `FlowersController` invokes `FlowerService.CreateAsync()`. The Gateway Service issues an HTTP POST with a JSON body to `localhost:5001/api/flowers`. The `CatalogService` API Controller receives the payload, constructs a `CreateFlowerCommand`, and passes it to the `CommandDispatcher`. The Dispatcher routes it to `CreateFlowerCommandHandler`, which persists the entity in MySQL and returns the new ID. The HTTP response is returned to the Gateway, which redirects the user.

3 Implementation Phase - Application

3.1 Tools and Technologies

The application continues to use the previous technology stack, heavily extended with network and architecture tools:

- **Architecture**: Microservices, API Gateway, CQRS

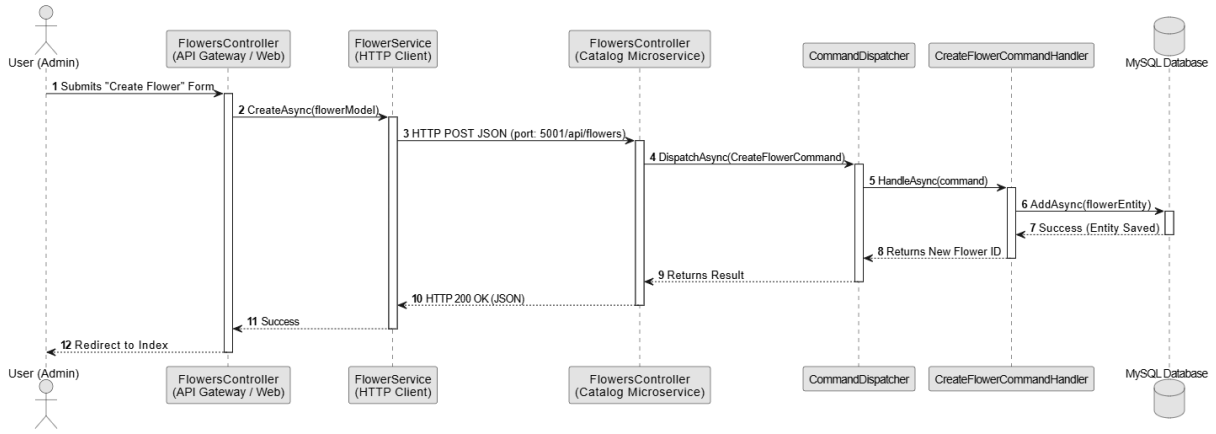


Figure 6: Sequence Diagram - Microservice API Gateway Flow

- **Frameworks**: ASP.NET Core 8.0 MVC (Gateway), ASP.NET Core 8.0 Web API (Microservices)
- **Inter-Service Communication**: `System.Net.Http.IHttpClientFactory` for robust, pooled HTTP connections
- **Serialization**: `System.Text.Json` with cycle ignoring mechanisms
- **Testing Framework**: **xUnit** paired with **Microsoft.EntityFrameworkCore.InMemory** provider

3.2 Application Structure

The solution structure has been radically decentralized into 5 distinct `.csproj` projects:

- **FlowerShop.Core**: Shared library containing `/CQRS` (Dispatchers, Interfaces), `/Models` (Domain Entities), and `/Export` (Template classes).
- **FlowerShop.CatalogService**: Contains `/CQRS/Commands`, `/CQRS/Queries`, and `/CQRS/Handlers` for flowers. Exposes endpoints via `/Controllers/FlowersController.cs`.
- **FlowerShop.OrderService**: Contains CQRS logic and endpoints for Orders.
- **FlowerShop.UserService**: Contains CQRS logic and endpoints for Users.
- **Software_Design1 (Gateway)**: Contains the `/Views`, authentication middleware, and `/Services` configured as HTTP clients.

3.3 Testing and Validation

A completely new suite of **unit tests** was implemented to validate the distributed CQRS architecture and the Template pattern, utilizing **xUnit** and **Microsoft.EntityFrameworkCore.InMemory**.

Testing ensures absolute business logic accuracy without requiring an active MySQL instance:

- **Command Handlers**: Tests verify that passing a `CreateFlowerCommand` to its handler successfully adds exactly one record to the In-Memory database, verifying stock, price, and category relationships.
- **Query Handlers**: Tests verify that `GetAllFlowersQuery` and `GetOrderByIdQuery` retrieve accurate data sets, properly applying mapping and filtering conditions.
- **Template Export**: Tests verify that calling `Export()` on the `JsonExporter` correctly invokes the template algorithm, yielding valid JSON strings.

The entire test suite consists of 11 comprehensive tests. Running `dotnet test` yields a 100% pass rate, confirming the functional stability and architectural integrity of the newly decoupled microservices.